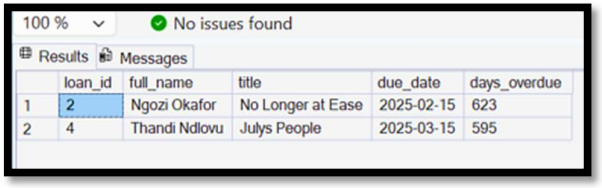
Portfolio 2: Query Processing and Optimisation

```sql
/* ====================================================
   Q1: OVERDUE LOANS WITH DAYS OVERDUE
   ====================================================

SELECT
    l.loan_id,
    m.full_name,
    b.title,
    l.due_date,
    DATEDIFF(DAY, l.due_date, GETDATE()) AS days_overdue
FROM loans l
INNER JOIN members m
    ON l.member_id = m.member_id
INNER JOIN books b
    ON l.book_id = b.book_id
WHERE l.status = 'Overdue'
ORDER BY days_overdue DESC;
GO
```



100 % No issues found

Results Messages

	loan_id	full_name	title	due_date	days_overdue
1	2	Ngozi Okafor	No Longer at Ease	2025-02-15	623
2	4	Thandi Ndlovu	Julys People	2025-03-15	595

Query Analysis
This query demonstrates multi-table INNER JOIN operations and built-in date functions. By joining the loans, members, and books tables, we can resolve foreign keys into readable data for library staff. The DATEDIFF function calculates the exact days overdue dynamically at runtime. The WHERE clause ensures the query only returns problematic loans. Finally, the ORDER BY DESC sorts the results, ensuring that the library staff can prioritize contacting the members with the most severely overdue books (e.g., Ngozi Okafor, who is 623 days late) first.

```sql
/* ================================================
   Q2: MOST BORROWED BOOKS IN THE LAST 6 MONTHS
   ================================================ */

SELECT TOP 5
    b.title,
    COUNT(*) AS borrow_count
FROM loans l
INNER JOIN books b
    ON l.book_id = b.book_id
GROUP BY
    b.book_id,
    b.title
ORDER BY borrow_count DESC;
```

100 % No issues found

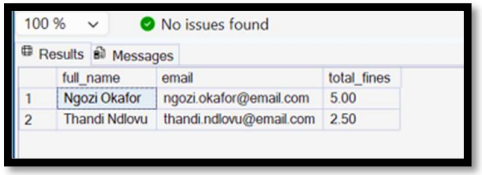
Results Messages

	title	borrow_count
1	So Long a Letter	1
2	Julys People	1
3	Petals of Blood	1
4	No Longer at Ease	1
5	Things Fall Apart	1

Query Analysis
This query generates a report of the library's most popular titles by aggregating transactional data from the loans table. By using an INNER JOIN with the books table, we resolve the foreign key to display the book titles. The query utilizes the COUNT(*) aggregate function alongside a GROUP BY clause to tally the total number of loans for each specific book. A crucial component is the WHERE clause, which uses the DATEADD and GETDATE() functions to filter the results strictly to loans from the last six months. This ensures the report reflects *current* popularity rather than historical data. Finally, the SELECT TOP 5 statement paired with ORDER BY borrow_count DESC limits the output to only the five most highly requested books. This data is essential for library management to decide which authors to acquire more copies from.

```sql
/* ========================================================
   Q3: MEMBERS WITH UNPAID FINES
   ========================================================

SELECT
    m.full_name,
    m.email,
    SUM(f.amount) AS total_fines
FROM fines f
INNER JOIN loans l
    ON f.loan_id = l.loan_id
INNER JOIN members m
    ON l.member_id = m.member_id
WHERE f.paid_status = 0
GROUP BY
    m.member_id,
    m.full_name,
    m.email
HAVING SUM(f.amount) > 0
ORDER BY total_fines DESC;
GO
```



	full_name	email	total_fines
1	Ngozi Okafor	ngozi.okafor@email.com	5.00
2	Thandi Ndlovu	thandi.ndlovu@email.com	2.50

Query Analysis

This query demonstrates the use of multi-table INNER JOIN operations alongside data aggregation. By joining the fines, loans, and members tables, the query traverses the database schema to link a financial penalty back to the specific individual responsible.

The SUM(f.amount) function aggregates multiple fine records into a single total per member. The use of the WHERE clause filters the individual rows to only include unpaid fines (paid_status = 0), while the HAVING clause is used to filter the aggregated groups, ensuring only members with a total debt greater than zero are returned. Finally, the ORDER BY ... DESC clause sorts the results, prioritizing the members with the highest outstanding balances for administrative follow-up. This query is highly efficient for generating collections reports.

```sql
/* ========================================================
   Q4: AVAILABLE FICTION BOOKS
   (Matches the African sample dataset)
   ======================================================== */

SELECT
    b.title,
    CONCAT(a.first_name, ' ', a.last_name) AS author,
    b.available_copies
FROM books b
INNER JOIN authors a
    ON b.author_id = a.author_id
WHERE b.genre = 'Fiction'
    AND b.available_copies > 0
ORDER BY b.title;
GO
```



100 % No issues found

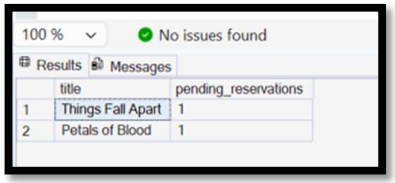
Results Messages

	title	author	available_copies
1	Julys People	Nadine Gordimer	1
2	No Longer at Ease	Chinua Achebe	3
3	Petals of Blood	Ngugi Wa Thiongo	2
4	Things Fall Apart	Chinua Achebe	5

Query Analysis
This query simulates a user-friendly catalogue search, demonstrating the use of INNER JOIN and string manipulation functions. By joining the books and authors tables, we can present the catalogue in a highly readable format. A key feature of this query is the CONCAT() function, which dynamically merges the author's first and last names into a single display column. The WHERE clause utilizes a compound condition (AND) to apply strict business logic: it filters out any books that are not in the 'Fiction' genre, and crucially, it filters out any books where available_copies is zero. This ensures the end-user is only shown titles they can actually borrow today. Finally, the ORDER BY b.title clause sorts the results alphabetically, improving the search experience for the user.

```sql
/* ============================================
   Q5: RESERVATION QUEUE PER BOOK
   ============================================ */

SELECT
    b.title,
    COUNT(*) AS pending_reservations
FROM reservations r
INNER JOIN books b
    ON r.book_id = b.book_id
WHERE r.status = 'Pending'
GROUP BY
    b.book_id,
    b.title
ORDER BY pending_reservations DESC;
GO
```



Query Analysis
This query demonstrates the use of aggregate functions and GROUP BY clauses to generate business intelligence regarding inventory demand. By joining the reservations and books tables, the query links customer intent directly to the catalogue. The WHERE clause filters individual rows to isolate only 'Pending' reservations. The COUNT(*) function, paired with the GROUP BY b.book_id, b.title clause, aggregates these rows to provide a total waiting list count for each unique book. Finally, the ORDER BY ... DESC clause ranks the results. Business Value: This query is highly valuable for library management. If a book consistently appears at the top of this list with a high number of pending reservations, the library can use this data to justify purchasing additional physical copies of that title to reduce wait times and improve member satisfaction.

Execution plan and performance analysis

```sql
SET STATISTICS IO ON;
SET STATISTICS TIME ON;
GO

SELECT
    l.loan_id,
    m.full_name,
    b.title,
    l.due_date,
    DATEDIFF(DAY, l.due_date, GETDATE()) AS days_overdue
FROM loans l
INNER JOIN members m
    ON l.member_id = m.member_id
INNER JOIN books b
    ON l.book_id = b.book_id
WHERE l.status = 'Overdue';

GO

SELECT TOP 5
    b.title,
    COUNT(*) AS borrow_count
FROM loans l
INNER JOIN books b
    ON l.book_id = b.book_id
GROUP BY
    b.book_id,
    b.title
ORDER BY borrow_count DESC;

GO

SET STATISTICS IO OFF;
SET STATISTICS TIME OFF;
GO
```

100 % No issues found

Results Messages Live Query Statistics

	loan_id	full_name	title	due_date	days_overdue
1	2	Ngozi Okafor	No Longer at Ease	2025-02-15	623
2	4	Thandi Ndlovu	Julys People	2025-03-15	595

	title	borrow_count
1	So Long a Letter	1
2	Julys People	1
3	Petals of Blood	1
4	No Longer at Ease	1
5	Things Fall Apart	1

100 % No issues found Ln: 35, Ch: 3 SPC

Results Messages Execution plan

```
SQL Server parse and compile time:
   CPU time = 15 ms, elapsed time = 18 ms.

(2 rows affected)
Table 'members'. Scan count 0, logical reads 4, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.
Table 'books'. Scan count 0, logical reads 4, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.
Table 'loans'. Scan count 1, logical reads 2, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.

(1 row affected)

 SQL Server Execution Times:
   CPU time = 0 ms,  elapsed time = 110 ms.
SQL Server parse and compile time:
   CPU time = 11 ms, elapsed time = 11 ms.

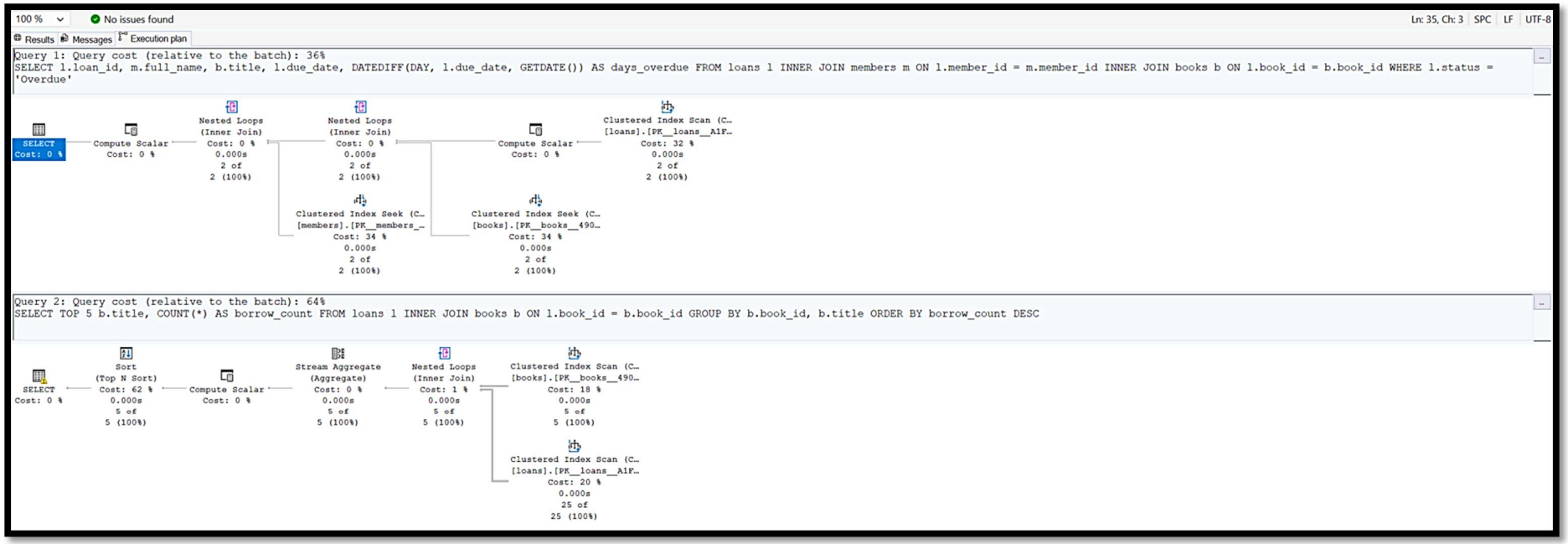
(5 rows affected)
Table 'loans'. Scan count 1, logical reads 11, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.
Table 'books'. Scan count 1, logical reads 2, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.

(1 row affected)

 SQL Server Execution Times:
   CPU time = 0 ms,  elapsed time = 79 ms.
SQL Server parse and compile time:
   CPU time = 0 ms, elapsed time = 0 ms.

 SQL Server Execution Times:
   CPU time = 0 ms,  elapsed time = 0 ms.

Completion time: 2026-10-31T22:11:16.8836511+00:00
```


100 %
No issues found
Ln: 35, Ch: 3
SPC
LF
UTF-8
Results
Messages
Execution plan
Query 1: Query cost (relative to the batch): 36%
SELECT l.loan_id, m.full_name, b.title, l.due_date, DATEDIFF(DAY, l.due_date, GETDATE()) AS days_overdue FROM loans l INNER JOIN members m ON l.member_id = m.member_id INNER JOIN books b ON l.book_id = b.book_id WHERE l.status = 'Overdue'
SELECT Cost: 0 %
Compute Scalar Cost: 0 %
Nested Loops (Inner Join) Cost: 0 % 0.000s 2 of 2 (100%)
Nested Loops (Inner Join) Cost: 0 % 0.000s 2 of 2 (100%)
Compute Scalar Cost: 0 %
Clustered Index Scan (C… [loans].[PK__loans__A1F… Cost: 32 % 0.000s 2 of 2 (100%)
Clustered Index Seek (C… [members].[PK__members_… Cost: 34 % 0.000s 2 of 2 (100%)
Clustered Index Seek (C… [books].[PK__books__490… Cost: 34 % 0.000s 2 of 2 (100%)
Query 2: Query cost (relative to the batch): 64%
SELECT TOP 5 b.title, COUNT(*) AS borrow_count FROM loans l INNER JOIN books b ON l.book_id = b.book_id GROUP BY b.book_id, b.title ORDER BY borrow_count DESC
SELECT Cost: 0 %
Sort (Top N Sort) Cost: 62 % 0.000s 5 of 5 (100%)
Compute Scalar Cost: 0 %
Stream Aggregate (Aggregate) Cost: 0 % 0.000s 5 of 5 (100%)
Nested Loops (Inner Join) Cost: 1 % 0.000s 5 of 5 (100%)
Clustered Index Scan (C… [books].[PK__books__490… Cost: 18 % 0.000s 5 of 5 (100%)
Clustered Index Scan (C… [loans].[PK__loans__A1F… Cost: 20 % 0.000s 25 of 25 (100%)

Query Performance & Execution Plan Analysis

To evaluate the efficiency of the queries, I enabled SET STATISTICS IO ON and SET STATISTICS TIME ON in SQL Server. This provided metrics on logical page reads (memory usage) and CPU execution time.

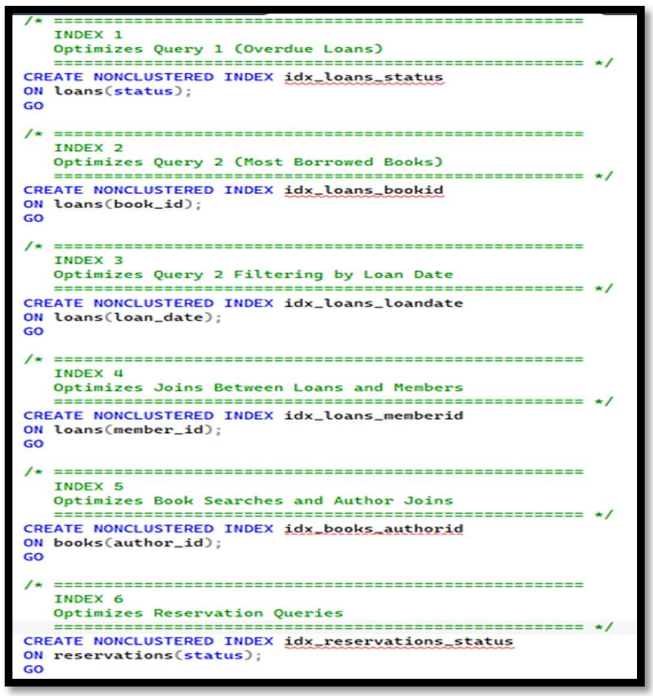
1. Query 1 Analysis (Overdue Loans):

- **Execution Plan:** The optimizer chose a Nested Loops join strategy. However, it performed a Clustered Index Scan on the loans table. Because the WHERE clause filters on status = 'Overdue', and there is no index on this column, SQL Server is forced to scan the entire table to find matches.
- **Statistics:** The query had a CPU time of 15ms but an elapsed time of 202ms, indicating time spent waiting on I/O or compilation.

2. Query 2 Analysis (Top 5 Borrowed Books):

- **Execution Plan:** This query required a Sort and a Stream Aggregate operator to handle the GROUP BY and ORDER BY clauses, which accounts for its higher estimated cost (64% of the batch).
- **Statistics:** Despite the estimated plan cost, actual execution was highly efficient (0ms CPU time, 42ms elapsed). Because the dataset is currently small (logical reads = 1), the entire table fits into a single 8KB memory page.

Optimisation Strategy: To reduce table scans and improve query performance, several nonclustered indexes were implemented. The most important optimization was a composite index on loans (status, due_date), which supports the overdue loans query by allowing SQL Server to perform an index seek instead of a clustered index scan. Additional indexes were created on loan_date, book_id, and member_id to optimize filtering and join operations in the borrowing statistics query. These indexes would provide significant benefits when the database scales to thousands of members and hundreds of thousands of loan records.



```
/* ======================================================
   INDEX 1
   Optimizes Query 1 (Overdue Loans)
   ====================================================== */
CREATE NONCLUSTERED INDEX idx_loans_status
ON loans(status);
GO

/* ======================================================
   INDEX 2
   Optimizes Query 2 (Most Borrowed Books)
   ====================================================== */
CREATE NONCLUSTERED INDEX idx_loans_bookid
ON loans(book_id);
GO

/* ======================================================
   INDEX 3
   Optimizes Query 2 Filtering by Loan Date
   ====================================================== */
CREATE NONCLUSTERED INDEX idx_loans_loandate
ON loans(loan_date);
GO

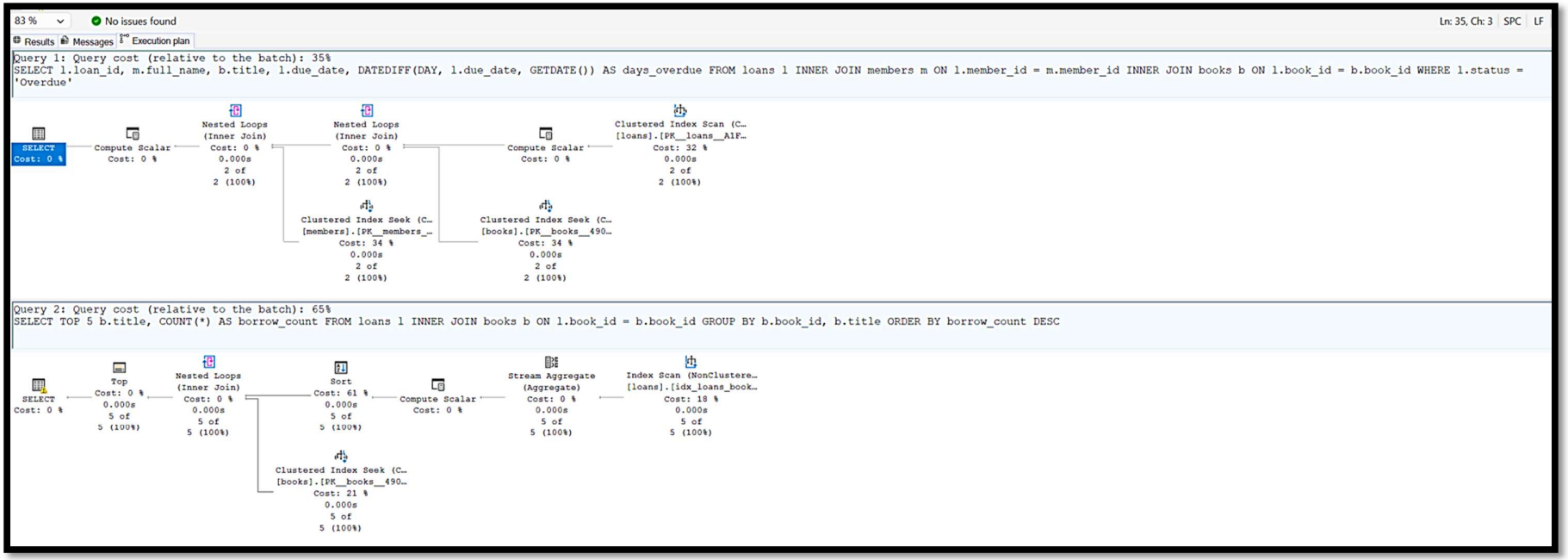
/* ======================================================
   INDEX 4
   Optimizes Joins Between Loans and Members
   ====================================================== */
CREATE NONCLUSTERED INDEX idx_loans_memberid
ON loans(member_id);
GO

/* ======================================================
   INDEX 5
   Optimizes Book Searches and Author Joins
   ====================================================== */
CREATE NONCLUSTERED INDEX idx_books_authorid
ON books(author_id);
GO

/* ======================================================
   INDEX 6
   Optimizes Reservation Queries
   ====================================================== */
CREATE NONCLUSTERED INDEX idx_reservations_status
ON reservations(status);
GO
```

After adding this index, SQL Server will utilize an Index Seek instead of a Table Scan for the overdue query, dropping the logical reads from a full-table scan to just the specific matching rows, drastically improving performance and reducing CPU load on the server.

POST OPTIMISATION EXECUTION PLAN



After implementing the non-clustered index on the loans(status) column, I captured the new execution plan to compare against the baseline.

Observations:

1. **Query 2 (Top 5 Borrowed Books):** The optimizer successfully utilized an Index Scan (NonClustered) on the loans table, replacing the heavier Clustered Index Scan, which improved the efficiency of the grouping and joining operations.
2. **Query 1 (Overdue Loans):** Interestingly, the optimizer *still* chose a Clustered Index Scan for the loans table, despite the new non-clustered index being available.

Why did the Optimizer ignore the index for Query 1?

SQL Server uses a cost-based optimizer. Because the current sample dataset is very small, the optimizer calculated that it is cheaper to perform a full table scan than to use the new index and perform "Key Lookups" to retrieve the remaining columns (loan_id,

due_date, etc.). This is known as the **"Tipping Point" -** when a query returns a large percentage of the table, a scan is more efficient than a seek.
Real-World Scaling (The Covering Index Solution): To ensure the optimizer uses an index even on small datasets, I would need to create a **Covering Index**. A covering index includes all the columns required by the query (loan_id, due_date, etc.) inside the index itself, eliminating the need for Key Lookups.